

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа

Выполнил:

Свинин Артем

Группа К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Цель работы

Развернуть микросервисное приложение на удалённом сервере: провести предварительную настройку окружения, выполнить деплой через Docker Compose и настроить Nginx в режиме обратного прокси для внешнего доступа.

Что реализовано

- Написан скрипт `bootstrap.py`, который автоматизирует предварительную настройку сервера и первичный деплой приложения.
- На сервере устанавливаются все необходимые зависимости: Docker, Docker Compose plugin, Nginx, Certbot, fail2ban.
- Настроен фаервол UFW: открыты только порты SSH и Nginx Full (80/443).
- Настроен fail2ban для защиты SSH от брутфорса.
- Реализована структура каталогов с поддержкой нескольких релизов и автоматическим откатом.
- Nginx настроен в режиме обратного прокси.
- При наличии домена автоматически выпускается TLS-сертификат через Certbot.
- Настроен systemd-юнит для автозапуска приложения после перезагрузки сервера.

Скрипт `provision_server.py`

Скрипт запускается один раз на чистом сервере от имени `root` и последовательно выполняет все шаги подготовки и деплоя.

Запуск

```
sudo python3 provision_server.py git@github.com:ORG/REPO.git \  
--branch main \  
--user deploy \  
--dir /opt/rental-platform \  
--port 8080 \  
--domain example.com \  
--email admin@example.com
```


Аргументы командной строки:

Аргумент	По умолчанию	Описание
repo	—	URL Git-репозитория (обязательный)
--branch	main	Ветка для деплоя
--user	deploy	Пользователь, от которого делается git clone
--dir	/opt/rental-platform	Каталог приложения на сервере
--port	8080	Порт, на котором слушает приложение
--domain	—	Домен (если задан — выпускается SSL)
--email	—	Email для Certbot

Шаг 1 — Установка системных пакетов (install_packages)

```
run(["apt", "update"])
run(["apt", "upgrade", "-y"])
run(["apt", "install", "-y",
    "curl", "git", "ufw", "nginx", "fail2ban",
    "ca-certificates", "gnupg", "lsb-release",
    "certbot", "python3-certbot-nginx"])
```

Устанавливаются все зависимости, необходимые для дальнейших шагов: инструменты загрузки (curl), система контроля версий (git), веб-сервер (nginx), защита (ufw, fail2ban) и инструменты для получения TLS-сертификатов (certbot, python3-certbot-nginx).

Шаг 2 — Установка Docker (install_docker)

if shutil.which("docker") is None:

```
run(["curl", "-fsSL", "https://get.docker.com", "-o", "/tmp/get-docker.sh"])
```

```
run(["sh", "/tmp/get-docker.sh"])
```

```
run(["rm", "-f", "/tmp/get-docker.sh"])
```

```
run(["apt", "install", "-y", "docker-compose-plugin"], check=False)
```

```
run(["systemctl", "enable", "--now", "docker"], check=False)
```

Перед установкой проверяется, не установлен ли Docker уже (shutil.which("docker")).

Используется официальный установочный скрипт get.docker.com. После установки включается docker-compose-plugin и Docker добавляется в автозапуск через systemd.

Шаг 3 — Настройка фаервола (configure_firewall)

```
run(["ufw", "allow", "OpenSSH"])
```

```
run(["ufw", "allow", "Nginx Full"])
```

```
run(["ufw", "--force", "enable"])
```

UFW настраивается так, что разрешены только два профиля: OpenSSH (порт 22) и Nginx Full (порты 80 и 443). Весь остальной входящий трафик закрыт.

Шаг 4 — Настройка fail2ban (configure_fail2ban)

```
Path("/etc/fail2ban/jail.d/sshd.local").write_text(
```

```
"[sshd]\nenabled = true\nmaxretry = 5\n"
```

```
)
```

```
run(["systemctl", "enable", "--now", "fail2ban"])
```

Создается конфигурационный файл для jail sshd с лимитом в 5 неудачных попыток входа. fail2ban включается и запускается через systemd.

Шаг 5 — Деплой приложения (deploy)

Это основная функция скрипта. Она выполняет несколько этапов:

Каталог приложения /opt/rental-platform/ содержит три элемента. Папка releases/ хранит все задеплойенные версии — каждая в отдельном подкаталоге с именем в формате UTC-timestamp (например 20250510120000). Папка shared/ содержит файлы, общие для всех релизов — в первую очередь .env с переменными окружения. Симлинк current

указывает на активный релиз и переключается только после успешной проверки здоровья приложения.

Каталог `shared/` создаётся один раз. Если файл `.env` ещё не существует, скрипт ищет его в корне репозитория (`project/.env` или `project/.env.example`) и копирует в `shared/`.

Каталог `shared/` создаётся один раз. Если файл `.env` ещё не существует, скрипт ищет его в корне репозитория (`project/.env` или `project/.env.example`) и копирует в `shared/`.

Клонирование репозитория

```
ts = datetime.now(timezone.utc).strftime("%Y%m%d%H%M%S")
```

```
new_release = releases / ts
```

```
run(["sudo", "-u", user, "git", "clone", "--branch", branch, repo, str(new_release)])
```

Каждый деплой создаёт новый каталог с именем в формате UTC-timestamp, например 20250515093000. Клонирование выполняется от имени непривилегированного пользователя `deploy`.

Запуск контейнеров

```
run([docker, "compose",  
  
    "--env-file", str(env_file),  
    "-p", "rental-platform",  
    "-f", str(compose),  
    "up", "-d", "--build"]])
```

Docker Compose запускается с указанием файла `.env` из `shared/` и `docker-compose.yml` из нового релиза. Флаг `--build` гарантирует пересборку образов.

Health check и автоматический откат

```
for i in range(12):
```

```
    if run(["curl", "-fsS", f"http://localhost:{port}/health"], check=False) == 0:  
        healthy = True  
        break  
    time.sleep(5)
```

После запуска скрипт опрашивает эндпоинт /health — до 12 раз с интервалом 5 секунд (итого до 60 секунд на раскрутку). Если за это время приложение не ответило:

```
if not healthy:
    run([docker, "compose", ..., "down"])
    if old:
        run(["ln", "-sfn", str(old), str(current)])
    return
```

Новые контейнеры останавливаются, а симлинк `current` возвращается на предыдущий рабочий релиз.

При успехе симлинк переключается на новый релиз только после прохождения проверки:

```
run(["ln", "-sfn", str(new_release), str(current)])
```

Шаг 6 — Настройка Nginx (`configure_nginx`)

```
cfg = f"""
server {{
    listen 80;
    server_name {server_name};
    location / {{
        proxy_pass http://localhost:{port};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }}
}}

"""
```

```
Path("/etc/nginx/sites-available/app").write_text(cfg)
run(["ln", "-sfn", "/etc/nginx/sites-available/app", "/etc/nginx/sites-enabled/app"])
run(["rm", "-f", "/etc/nginx/sites-enabled/default"])
run(["nginx", "-t"])
run(["systemctl", "reload", "nginx"])
```

Nginx настраивается как обратный прокси: принимает входящие HTTP-запросы на порт 80 и перенаправляет их на localhost:8080, где слушает Nginx API Gateway внутри Docker. Дефолтный сайт Nginx (default) удаляется. Перед применением конфигурация проверяется командой `nginx -t`.

Шаг 7 — Выпуск TLS-сертификата (certbot)

```
if not domain or not email:
    logging.info("skip certbot")
    return
run(["certbot", "--nginx", "--non-interactive", "--agree-tos",
    "--redirect", "-m", email, "-d", domain])
```

Если при запуске скрипта переданы `--domain` и `--email`, Certbot автоматически получает сертификат Let's Encrypt и настраивает HTTPS в конфиге Nginx. Флаг `--redirect` добавляет перенаправление с HTTP на HTTPS. Если параметры не переданы — шаг пропускается.

Шаг 8 — Systemd-юнит (systemd)

```
Path("/etc/systemd/system/rental-platform.service").write_text(f"""
```

```
[Unit]
Description=Rental Platform
After=docker.service
Requires=docker.service
[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory={project / 'current'}
ExecStart={docker} compose -p rental-platform up -d --build
ExecStop={docker} compose -p rental-platform down

[Install]
WantedBy=multi-user.target
""")
run(["systemctl", "daemon-reload"])
run(["systemctl", "enable", "rental-platform"])
```

Создаётся системный юнит `rental-platform.service`. После перезагрузки сервера `systemd` автоматически выполняет `docker compose up -d --build` в каталоге `current`. При остановке сервиса выполняется `docker compose down`. Юнит запускается только после старта `docker.service`.